REPOPILOT · LITERATURE SYNTHESIS

# Project Understanding

Eleven papers from four team summaries, read against one question:
"Can I safely ship this vibe-coded app?"

From the summaries by Farag (test generation) · Hussein (coding agents) · Ziad (intent and harnesses) · Omar (repair and security)

# What RepoPilot is, and its research core

### THE SYSTEM

A QA platform for apps generated mostly by AI tools (Lovable, Bolt, Claude Code). Given a repository it:

- builds and runs the app in a sandbox,
- runs deterministic scanners and generated behavioural tests,
- decides what each failure means,
- applies a few bounded, verified fixes,
- writes an evidence-backed readiness report.

First target: React/Vite + Supabase + Stripe, e-commerce and CRUD apps.

### THE RESEARCH PROBLEM: ORACLES FOR CODE WITH NO SPEC

When a generated test fails, is the **test wrong** or the **code wrong**? Expectations inferred from the code itself copy the code's bugs.

RepoPilot infers intent from **independent sources**: README and docs, UI text, types and schemas, commit history, TODOs, agent artifacts (prompts, CLAUDE.md) and a verified reference-pattern library.

Each failure gets a verdict: `test_wrong`, `code_wrong`, `ambiguous` or `environment_issue`, with calibrated confidence and abstention.

Source: RepoPilot project brief, sections 1 to 4.

02 · THE READING MAP

# Eleven papers, four questions

| Summary | Paper | One-line contribution | RepoPilot question it informs |
|---|---|---|---|
| **Farag** Writing tests | CodeT5 (2021) | Identifier-aware pre-trained encoder-decoder for code | Which model ideas underlie test and patch generation? |
| | TestGen-LLM (Meta, 2024) | LLM tests kept only if they build, pass 5 times and add coverage | How do we filter generated tests? |
| | TestAgent (2026) | Planner, generator and reviewer agents over a knowledge graph | How do we plan tests and judge failures? |
| **Hussein** Running agents | CAT (ACL Findings 2026) | Context compression as a tool the agent calls | How do we keep long runs inside a budget? |
| | RepoMem (ICLR 2026) | Memory tools built from commit history for localisation | Is history useful evidence? |
| | SGAgent (arXiv 2026) | Localise, then suggest a repair plan, then fix | How should bounded repair be staged? |
| **Ziad** Intent and control | TiCoder (Microsoft) | User answers yes/no on generated tests to pin down intent | How do we resolve ambiguity? |
| | Agentic RAG (Apple) | Vector + graph retrieval with five agents for test artifacts | How should retrieval be built? |
| | Macedo | An agent harness = loop, tools, context and control | What is our system, and where is control? |
| **Omar** Repair and risk | RAP-Gen (FSE 2023) | Retrieve a similar past bug fix, then generate the patch with CodeT5 | How should the pattern library guide repair? |
| | (In)Security of Vibe-Coded Apps (2026) | 1,186 confirmed vulnerabilities in 200 deployed vibe-coded apps | What actually breaks in our target apps? |

Grouping is this synthesis's; paper details as reported in each team summary.

# From code models to verified, agentic tests

CODET5 · SALESFORCE · EMNLP 2021

T5 model trained on 8.35M code and comment instances. Learns which tokens are identifiers, and trains code-to-text and text-to-code together.

**43.20**

CodeBLEU on generation (PLBART 38.52); 65.78% defect detection

TESTGEN-LLM · META · FSE 2024

Extends Kotlin test classes. Each candidate must **build**, **pass 5 runs** (not flaky) and **add line coverage**, or it is discarded.

**75 / 57 / 25%**

built / passed reliably / added coverage; 73% of changes accepted by engineers

TESTAGENT · ZHANG ET AL. · 2026

Planner sets normal, boundary and error requirements; generator writes and runs tests; **three LLMs vote** on whether a failure is a real bug; reviewer uses coverage and mutation.

**154 / 167**

suspected failures were real bugs; mutation score 83.69% vs EvoSuite 43.59%

**Trajectory:** better models, then verified outputs, then autonomous multi-agent workflows. **Open limits:** coverage and mutation score are only proxies for test quality; results are single-run and benchmarks may overlap training data.

Source: Farag summary, slides 2 to 7.

# Context, memory and planning for coding agents

### CAT · FINDINGS OF ACL 2026

Context is split into a fixed segment, a summarised long-term memory and the last *k* steps. A fine-tuned 32B model learns when to compress.

**57.6%**

SWE-Bench-Verified, vs 49.8% plain ReAct and 53.8% fixed-threshold compression; context stays under 32k tokens

### REPOMEM · ICLR 2026

Episodic memory (past commits, diffs, issues) and semantic memory (summaries of the 200 most-edited files) as four tools added to LocAgent.

**76.5%**

Acc@5 file localisation (+4.9 over LocAgent), but **−13.1 points** on repos with under 10K commits

### SGAGENT · ARXIV, MAY 2026

Localiser finds up to 5 locations, suggester writes a repair plan with risks, fixer writes patches chosen by tests and voting. Shared repo knowledge graph.

**51.3%**

SWE-Bench-Lite; 38.0% without the suggest stage, 44.0% without the graph; $1.48 per issue

**Caution from the summary:** the three headline numbers use different tasks, models and benchmarks, so they cannot be compared with each other. No paper tests the three ideas together.

Source: Hussein summary, slides 2 to 10.

# Intent, retrieval and the agent harness

TICODER · LAHIRI ET AL., MICROSOFT

Natural language is ambiguous. The LLM generates code and tests; the user answers YES, NO or UNDEFINED on one test; contradicting code is pruned.

**48 → 71%**

MBPP pass@1 after one query (HumanEval 30.5 → 55.3%). User simulated; no user study

AGENTIC RAG · HARIHARAN ET AL., APPLE

Vector search plus a graph with 15 edge types (Requires, Validates, Covers...), five specialised agents, and links from requirements to tests to results.

**65 → 95%**

accuracy, basic RAG to full agentic system. Self-reported, one SAP project, inconsistent figures

WHAT MAKES A HARNESS · MACEDO

An agent harness needs four things: **T1** agent loop, **T2** tool interface, **T3** task-aware context, **T4** control that does not depend on the model obeying.

**6 pass**

Claude Code, Codex CLI, Aider, Cline, OpenHands, SWE-agent. They differ most in control

Source: Ziad summary, slides 2 to 10.

# Guided repair and vibe-app security

RAP-GEN · SALESFORCE · ESEC/FSE 2023

A hybrid retriever (BM25 word overlap + CodeT5 dense meaning) finds the closest past bug-fix pair; CodeT5 writes the patch with it as a hint.

**54.15**

TFix exact match vs 49.58 for a 3.5× larger T5; 125 Defects4J bugs fixed vs 110

A **random** hint (52.98) scored below **no** hint (53.46).

(IN)SECURITY OF VIBE-CODED APPS · DENG, FAN, MENG · 2026

9,041 apps built by Claude Code or Lovable; 200 deployed apps audited by agents, then confirmed by two experts (κ = 0.87).

**91%**

of apps have a vulnerability; 65.77% are Critical or High

**60.2%**

are broken access control, injection or auth failures

**20.7%**

of insecure runs: agent saw the risk and shipped anyway

Causes: memory, objective and knowledge defects. The largest single mode is **hidden security rules** (55.5%): requirements nobody stated.

Stack match: 96% of the apps are web apps on React and Vite; 55% use Supabase. The known Lovable flaw CVE-2025-48757 (CVSS 9.3) is missing Row-Level Security, one of RepoPilot's planned patch classes.

Source: Omar summary, slides 2 to 10.

07 · CROSS-CUTTING LESSON 1

# Execution is the evidence

Every paper that produces code or tests trusts it only after checking it outside the model.

| Paper | How output is accepted |
|---|---|
| TestGen-LLM | Build, pass 5 runs, add coverage |
| TestAgent | Run tests; coverage and mutation review |
| SGAgent | Candidate patches chosen by tests and voting |
| RAP-Gen | Patches checked by linter, unit tests or manual review |
| Vibe-coded security | Agent findings confirmed by exploit probe and two humans |
| Macedo (T4) | Control must not rely on the model obeying |

### The gap RepoPilot fills

TestGen-LLM **throws away failing tests** because it has no oracle. Those failures are exactly what RepoPilot must explain: test wrong or code wrong.

### Knowing is not doing

TestAgent's three voters all read the same code, and vibe-coding agents shipped insecure code in 20.7% of insecure runs even after naming the risk. Checks must block, not warn.

Synthesis across all four summaries; maps to brief principle "Execution is the evidence".

# Intent has to come from outside the code

TESTS AS INTENT

### TiCoder

One yes/no on a concrete test raised pass@1 by about 22 points.

**For us:** turn `ambiguous` verdicts into a short yes/no question to the owner.

HISTORY AS INTENT

### RepoMem

Commit memory helps on rich histories but lost 13.1 points on thin ones.

**For us:** vibe apps are built in a median of 8.39 days, so history is a weak signal.

REQUIREMENTS AS INTENT

### TestAgent, Agentic RAG

Write normal, boundary and error requirements first; link requirements to tests to results.

**For us:** store the evidence behind each test in `intent_evidence`.

UNSTATED INTENT

### Vibe-coded security

55.5% of vulnerabilities break rules nobody wrote down; TODOs and CLAUDE.md held real evidence.

**For us:** domain rules in the pattern library fill what docs never say.

Together these support the project's core bet: combining independent sources (docs, UI text, schemas, history, agent artifacts, the pattern library) should beat a judge that only reads the code. That is research questions 1 and 2.

"For us" lines are this synthesis's reading, not claims made by the papers.

09 · CROSS-CUTTING LESSON 3

# Retrieval helps only when it is good

### Evidence that retrieval and structure help

- **RAP-Gen:** hybrid word + meaning retrieval beat either alone (54.15 vs 53.88 / 53.93).
- **Agentic RAG:** 65.2% basic RAG → 87.1% hybrid vector-graph → 94.8% agentic.
- **SGAgent:** removing the repo knowledge graph drops 51.3% to 44.0%.
- **TestAgent:** a test knowledge graph lifts line coverage to 92.34% (HITS 82.39%).

### Evidence that it can hurt or mislead

- **RAP-Gen:** a random retrieved fix did worse than no retrieval (52.98 vs 53.46).
- **RepoMem:** memory lost 13.1 points on repos with thin history.
- **Agentic RAG:** self-reported, one enterprise project, conflicting figures.
- **CAT:** its summaries are never verified, so errors in them can persist.

**For the pattern library:** keep it small (10 to 20 verified patterns), measure retrieval quality on its own, always compare against "no retrieval", and keep benchmark apps out of it (RAP-Gen blocks the same leak in training). This is research question 4.

Synthesis across all four summaries; library rules from the project brief, section 5.

# Where each idea lands in RepoPilot

| RepoPilot stage | Idea we borrow | Source | What we must measure |
|---|---|---|---|
| Deterministic scanners | Prioritise access control (Supabase RLS), injection, auth, secrets | Vibe-coded security | Precision and recall per check family |
| Profile and risk map | Repo knowledge graph; history as a hint, verified in code | SGAgent, TestAgent, RepoMem | Localisation accuracy on young repos |
| Test plan and filter | Requirements first; build, pass repeatedly, add coverage | TestAgent, TestGen-LLM | Bug detection, false positives, mutation score |
| Failure attribution | Independent intent evidence; yes/no question when unsure | TiCoder, Agentic RAG | Verdict accuracy vs code-only judge; abstention |
| Bounded repair | Localise, plan, patch; retrieve a verified pattern as the hint | SGAgent, RAP-Gen | Repair success without regressions |
| Patch checks | Reject fixes that weaken a check (function-fix side effects) | Vibe-coded security | Weakened-check rate in proposed patches |
| Orchestration | Bounded context; harness with explicit control (T1–T4) | CAT, Macedo | Cost per app; run-to-run variance |

**Each stage must earn its place:** SGAgent's suggest stage is justified by an ablation (+13.3 points). Add a RepoPilot stage only when a measured ablation shows it helps.

**Budgets:** CAT stays under 32k tokens for 500 rounds; SGAgent costs $1.48 per issue. We cap tokens, time, retries and diff size, logged per app.

Synthesis mapping; principles and metrics from the project brief, sections 4 and 6.

11 · POSITIONING

# The gap, and what we must not repeat

### WHY IT MATTERS, AND THE OPEN GAP

Nine in ten deployed vibe-coded apps are vulnerable, and no prompt, model or harness setting removed the risk. Yet none of the eleven papers decides, with evidence, whether a failing test or the code is wrong for an app with **no specification**. Test generators discard or vote on failures; repair work assumes the bug is known; TiCoder needs a live user on one function; the security study audits but does not test behaviour. RepoPilot targets this gap with independent intent sources and calibrated abstention.

### WEAKNESSES THE SUMMARIES FLAG

- **Single runs** (TestAgent); 22% of vibe-coding replay cells changed outcome across identical runs: we repeat runs and report variance.
- **Simulated users** (TiCoder): we use real labels, two annotators, Cohen's κ.
- **Self-reported numbers** (Agentic RAG): we script and log every experiment.
- **Training-data overlap** (Farag): self-generated apps with saved prompts stay separate.
- **Proxy metrics** (coverage, exact match): we report true bugs found and repair without regressions.

Limits as stated in the four team summaries; responses from the project brief.